Module III

Syntax directed translation

* Associate information with a programming language construct by attaching attributes to the grammar symbols representing the construct.
* Values for attributes are computed by semantic rules associated with the grammar productions.
* There are two notations for associating semantic rules with productions, syntax directed definitions and translation schemes.
* Syntax-directed definitions are high level specifications for translations. They hide many implementation details and free the user from having to specify explicitly the order in which translation takes place.
* Translation schemes indicate the order in which semantic rules are to be evaluated, so they allow some implementation details to be shown.
* Conceptually, with both syntax-directed definitions and translation schemes, we parse the input token stream, build the parse tree

and then traverse the tree as needed to evaluate the semantic rules at the parse tree nodes

* The translation of the token stream is the result obtained by evaluating the semantic rules.

Syntax - Directed Definitions

* It is a generalization of a context free grammar in which each grammar symbol has an associated set of attributes, partitioned in to two subsets called the synthesized and inherited attributes of that grammar symbol.

* The value of an attribute at a parse tree node is defined by a semantic rule associated with the production used at that node.

* The value of a synthesized attribute at a node is computed from the values of attribute at the children of that node in the parse tree, the value of an inherited attribute is computed from the values of attribute at the siblings and parent of that node.

Form of a syntax directed definition.

* In a syntax-directed definition, each grammar production $A \to \alpha$ has associated with it a set of semantic rules of the form $b = f(c_1, c_2, \ldots c_k)$ where f is a function, and either,

(1) b is a synthesized attribute of A and $c_1, c_2, \ldots c_k$ are attributes belonging to the grammar symbols of the production $A \to XY$, $A.VAL \to X.VAL + Y.VAL$

or

(2) b is an inherited attribute of one of the grammar symbols on the right side of the production and $c_1, c_2, \ldots c_k$ are attributes belonging to the grammar symbol of the production.

$A \to XY$, $Y.VAL = 2 * A.VAL$.

* In a syntax directed definition, terminals are assumed to have synthesized attributes only, attributes of terminals are usually supplied by the ~~terminals~~ lexical analyzer.
* The start symbol is assumed not to have any inherited attributes, unless otherwise stated.

Synthesized translation

* The value of the translation of the nonterminal on the left side of the production as a function of the translations of the nonterminals on the right side. Such a translation is called a synthesized translation

→ $E \rightarrow E^{(1)} + E^{(2)}$ $E.VAL = E^{(1)}.VAL + E^{(2)}.VAL$.

Inherited translation

* The translation of a nonterminal on the rightside of the production is defined in terms of a translation of the nonterminal on the left. Such a translation is called an inherited translation.

→ $A \rightarrow XYZ$, $Y.VAL = 2 * A.VAL$.

Syntax Directed Definitions

* A syntax directed definition that uses synthesized attributes exclusively is said to be an S-attributed definition.

* Consider the grammar,

Production	Semantic rules
$L \rightarrow En$	Print (E.val)
$E \rightarrow E + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow digit$	$F.val = digit.lexval$

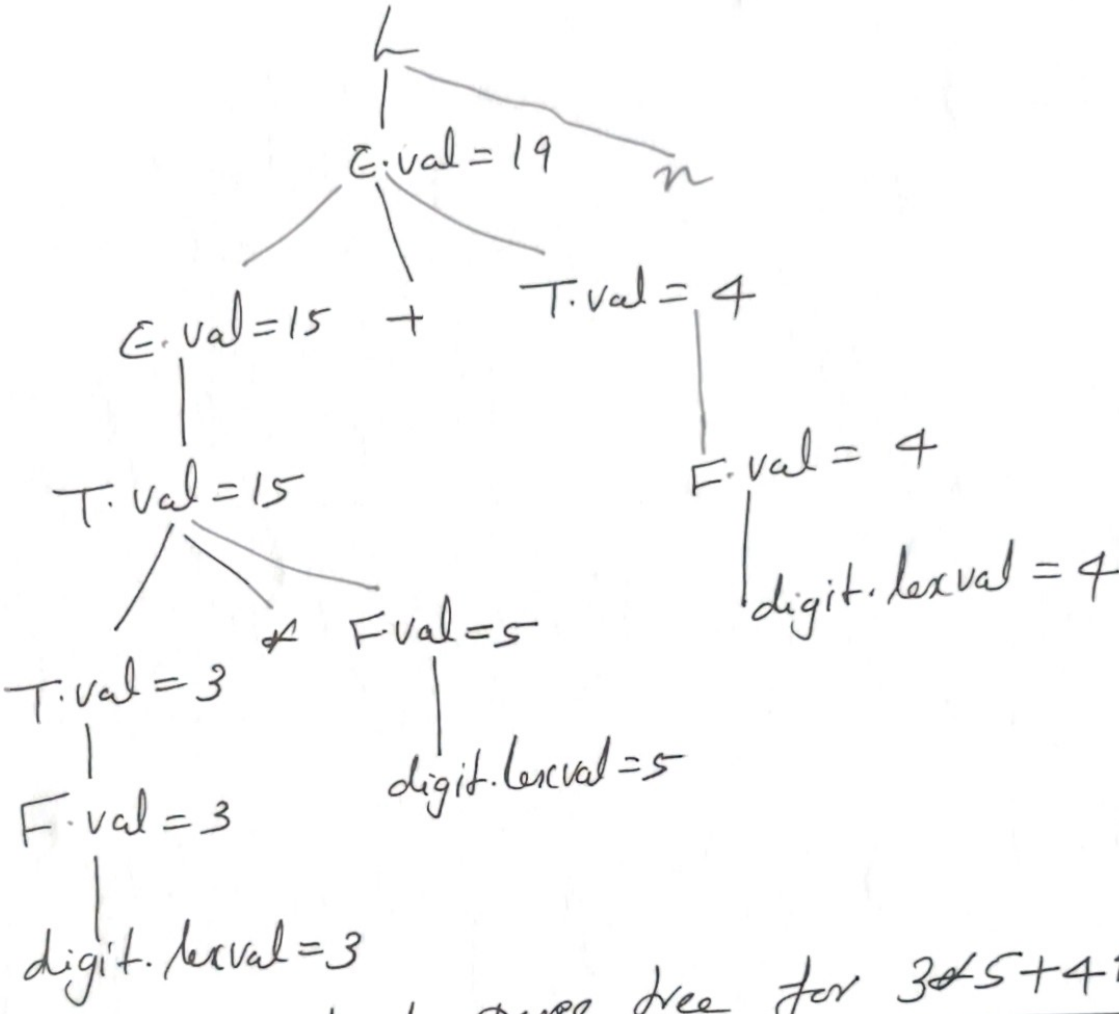

Fig. Annotated parse tree for 3*5+4n.

* A parse tree for an S-attributed definition can always be annotated by evaluating the semantic rules for the attributes at each node bottom-up, from the leaves to the root.

Syntax directed definition with inherited attribute

* Consider the grammar,

production	Semantic Rules
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

Fig. Syntax directed definition with inherited attribute L.in.

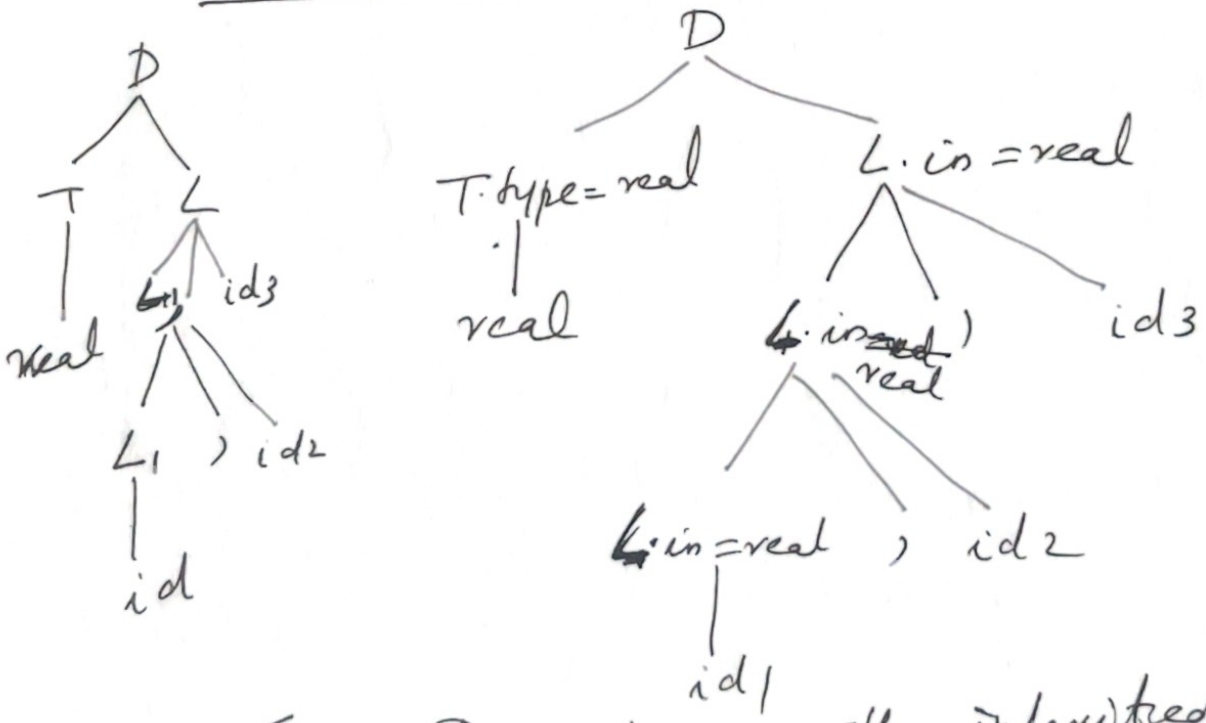

Fig. Parse tree with inherited attribute at each node labeled L.

Input string real id1, id2, id3